

Kafka Consumer Lag Root-Cause Analyzer

Project Report – Phase 1

1 Team Members

Rishi Kalra (23b1081), Avinash Chaudhari (23b1064), Tejas Chaudhari (23b0932), Hari Shankar Karthik (23b0907)

2 Problem Statement

Apache Kafka is widely used for real-time data pipelines. A critical issue in such systems is **consumer lag**, which indicates that consumers are unable to keep up with incoming data.

While existing monitoring tools can detect lag, they do not explain its underlying causes. Engineers must manually inspect multiple metrics, making debugging complex and time-consuming. This project aims to design a system that automatically analyzes and suggests possible root causes of consumer lag.

3 Understanding Causes of Consumer Lag

Consumer lag is the difference between the latest offset and the committed offset. It is only a symptom, and multiple underlying causes may lead to it:

- **Slow Consumer Processing:**
Occurs when the application processes messages slowly due to heavy computation or I/O.
Observation: Lag increases steadily and throughput decreases.
- **Partition Imbalance:**
Uneven distribution of data across partitions overloads some consumers.
Observation: High lag in a few partitions while others remain underutilized.
- **Broker Throttling** Brokers limit consumption rate due to quotas or protection mechanisms.
Observation: Increased latency and lag without failures.
- **Network Delays:**
High latency or packet loss affects communication between brokers and consumers.
Observation: Increased fetch latency and inconsistent throughput.
- **Commit Failures:**
Offsets are not committed properly despite message processing.
Observation: Lag remains high even though processing appears normal.
- **Rebalance Loops:**
Frequent consumer group rebalancing interrupts processing.
Observation: Lag spikes and temporary drop in throughput.

4 Objectives

- Collect Kafka consumer and broker metrics
- Correlate signals such as lag, throughput, and commits
- Identify probable root causes of lag
- Provide clear and interpretable diagnostic output

5 Proposed Solution

The proposed system will:

1. Collect metrics including lag, throughput, commit latency, rebalance events, partition load, and throttle time
2. Correlate these metrics over time using rule-based logic or simple ML models
3. Identify root causes such as slow consumers, imbalance, throttling, and failures
4. Output structured diagnostics

Consumer Group: payments-service

Lag: 120000

Root Cause: Partition Imbalance

Observation: Broker 2 has 3x traffic

6 High-Level Design

Metrics Collector: Collects data using Kafka APIs and JMX metrics.

Correlation Engine: Aligns time-series data and detects anomalies.

Root Cause Analyzer: Applies rules to infer causes.

Output Module: Displays results via CLI (extendable to dashboards).

7 Implementation Plan

- Use Kafka APIs for lag and metadata
- Use JMX/exporters for broker metrics
- Store data in lightweight storage
- Implement logic in Python or Java
- Build a simple CLI interface

8 Scope and Limitations

Scope: Focus on rule-based detection of common lag causes.

Limitations: May not handle all edge cases; accuracy depends on metrics; limited real-time capability.

9 Expected Outcomes

- Prototype system for lag detection and diagnosis
- Faster debugging of Kafka pipelines
- Basis for future extensions (ML, real-time systems)

10 Conclusion

This project addresses a gap in Kafka observability by moving from lag detection to root-cause analysis, improving debugging efficiency and system reliability.